

KLCP — Chapter 2 Practical Lab

Overview

This is a write-up of a hands-on Linux command-line lab I completed in a Kali Linux virtual machine while studying **Kali Linux Revealed (KLCP / PEN-103)** from OffSec. It is a self-completed learning exercise, not a certification submission.

The lab covers core shell skills: navigating the filesystem, creating and manipulating files, searching with `find` and `grep`, changing file permissions, running and inspecting a background process, collecting system information, and writing a small Bash script. Several commands failed along the way. I have kept those errors in the report because they show how I diagnosed and corrected mistakes.

All commands and outputs below come from my own screenshots. Where something is cut off or cannot be confirmed, I say so.

Learning Objectives

Skills demonstrated by the screenshots:

- Navigating the filesystem (`pwd`, `ls`, `cd`)
- Creating directories and files (`mkdir`, `echo` with redirection)
- Reading, copying and moving files (`cat`, `cp`, `mv`)
- Reading long-format listings (`ls -la`, `ls -l`)
- Searching for files and content (`find`, `grep -r`)
- Changing permissions with octal modes (`chmod 600 / 644 / 755`)
- Background jobs and process inspection (`&`, `jobs`, `ps aux`, `kill`)
- Querying system information (`id`, `uname`, `free`, `df`, `lspci`, `lsusb`, `dmesg`)
- Writing and running a Bash script, and debugging it from its error output

Environment

```
Operating System : Kali Linux (rolling), VirtualBox guest
VM name          : kali_LINUX (Oracle VirtualBox, state: Running)
Kernel           : Linux 6.19.14+kali-amd64 (x86_64), build dated 2026-05-05
Shell            : Appears to be zsh (see note below)
User             : kali (uid=1000)
Lab directory    : /home/kali/linux-lab
Date of lab      : Oct 5 (year not shown in the listings; screenshot filenames indicate 2026-10-05)
Course / Chapter : KLCP / PEN-103, Chapter 2
```

Shell note: the `dquote>` continuation prompt and the `cd: no such file or directory: linux-lab` error format are characteristic of zsh. The screenshots do not show `echo $SHELL`, so this is an inference.

The directory name `linux-lab` and the text strings "linus fundamental" and "kali linux" are reproduced exactly as typed.

Lab Walkthrough

Step 1 — Filesystem navigation and creating a lab directory

Commands

```
pwd
ls
mkdir linux-lab
ls
cd linux-lab
pwd
cd ..
```

What I did. I started in my home directory, checked where I was, listed its contents, created a new directory called `linux-lab`, confirmed it appeared, moved into it, confirmed the new path, and moved back up one level.

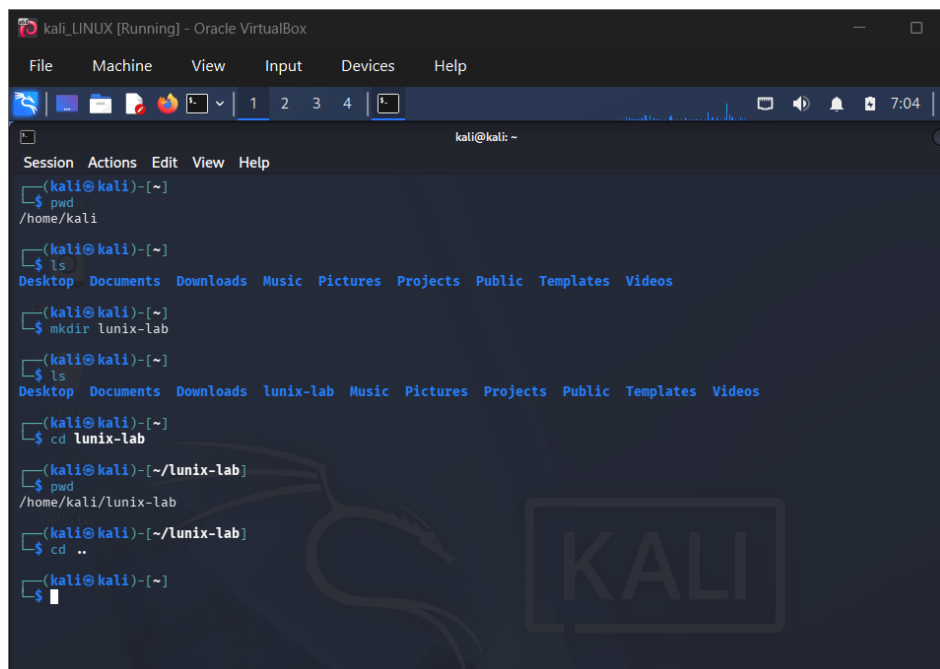
Result

- `pwd` returned `/home/kali`.
- The first `ls` showed `Desktop Documents Downloads Music Pictures Projects Public Templates Videos`.
- After `mkdir linux-lab`, the second `ls` included `linux-lab`.
- Inside the directory, `pwd` returned `/home/kali/linux-lab` and the prompt changed to `~/linux-lab`.
- `cd ..` returned me to `~`.

What I learned

- `pwd` prints the absolute path of the working directory.
- `ls` lists directory contents.
- `mkdir` creates a directory.
- `cd` changes directory, and `..` means the parent directory.
- The shell prompt reflects the current directory, which is a quick way to check location.

Evidence: `Screenshot_2026-10-05_140435.png`

A screenshot of a Kali Linux terminal window titled 'kali_LINUX [Running] - Oracle VirtualBox'. The terminal shows a series of commands and their outputs:
1. `pwd` returns `/home/kali`.
2. `ls` lists `Desktop Documents Downloads Music Pictures Projects Public Templates Videos`.
3. `mkdir linux-lab` creates the directory.
4. `ls` now includes `linux-lab` in the list.
5. `cd linux-lab` changes the directory.
6. The prompt changes to `(kali@kali)-[~/linux-lab]`.
7. `pwd` returns `/home/kali/linux-lab`.
8. `cd ..` returns to the home directory.
9. The prompt returns to `(kali@kali)-[~]`.
The terminal background features a faint Kali Linux logo and the word 'KALI' in large letters.

Step 2 — Creating, appending, copying and moving files

Commands

```
cd linux-lab
cd linux-lab          # fails, see Troubleshooting #1
ls
echo "linus fundamental">notes.txt
cat notes.txt
echo "kali linux">> notes.txt
cat notes.txt
cp notes.txt backup.txt
cat backup.txt
mv backup.txt notes.txt
cat notes.txt
ls -la
```

What I did. I re-entered the lab directory and created `notes.txt` by redirecting `echo` output with `>`. I appended a second line with `>>`, copied the file to `backup.txt`, and then used `mv` to rename `backup.txt` back to `notes.txt`. Finally I listed everything in long format.

Result

- `ls` printed nothing, so the directory was empty at that point.
- After `>`, `cat` showed `linus fundamental`.
- After `>>`, `cat` showed both `linus fundamental` and `kali linux`.
- `cat backup.txt` showed the same two lines, so the copy was identical.
- After `mv backup.txt notes.txt`, `cat notes.txt` still showed the same two lines.
- The final `ls -la` output:

```
total 12
drwxrwxr-x  2 kali kali 4096 Oct  5 07:08 .
drwx----- 18 kali kali 4096 Oct  5 07:03 ..
-rw-rw-r--  1 kali kali   29 Oct  5 07:08 notes.txt
```

Output analysis

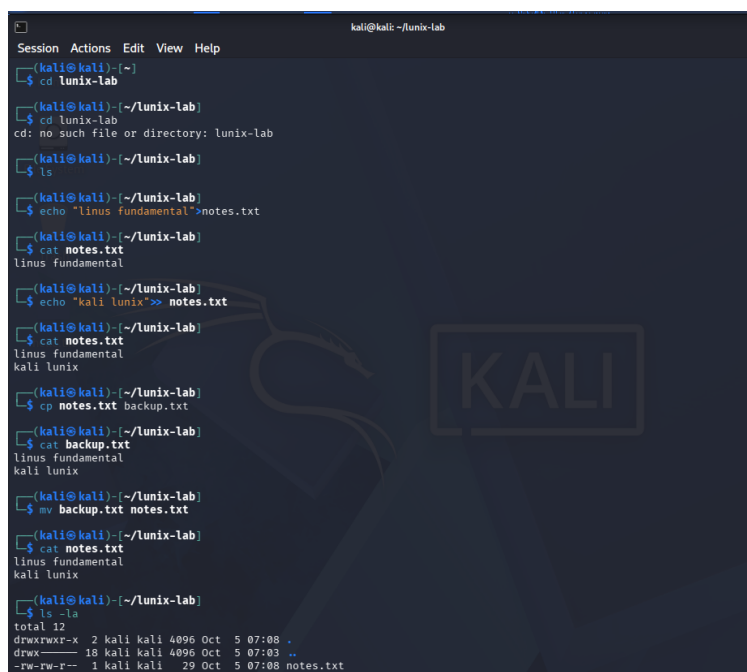
- `backup.txt` is gone from the listing. `mv` renamed it onto `notes.txt`, replacing the original without any prompt. Because the two files had identical content, the result looked unchanged, but the overwrite is the behaviour to remember.
- `notes.txt` is 29 bytes. That matches 18 bytes for `linus fundamental` plus a newline and 11 bytes for `kali linux` plus a newline.
- `.` is the lab directory (`drwxrwxr-x`) and `..` is my home directory (`drwx-----`, owner-only access).

Options used

- `ls -la`: `-l` gives long format, `-a` includes hidden entries (`.` and `..`).
- `>` overwrites a file and `>>` appends to it.

What I learned: redirection operators, the difference between copying and moving, that `mv` can silently overwrite, and how to read the columns of a long listing.

Evidence: [Screenshot_2026-10-05_140944.png](#)



```
kali@kali: ~/linux-lab
Session Actions Edit View Help
(kali@kali)~$ cd linux-lab
(kali@kali)~/linux-lab$ cd linux-lab
cd: no such file or directory: linux-lab
(kali@kali)~/linux-lab$ ls
(kali@kali)~/linux-lab$ echo "linus fundamental">notes.txt
(kali@kali)~/linux-lab$ cat notes.txt
linus fundamental
(kali@kali)~/linux-lab$ echo "kali linux">> notes.txt
(kali@kali)~/linux-lab$ cat notes.txt
linus fundamental
kali linux
(kali@kali)~/linux-lab$ cp notes.txt backup.txt
(kali@kali)~/linux-lab$ cat backup.txt
linus fundamental
kali linux
(kali@kali)~/linux-lab$ mv backup.txt notes.txt
(kali@kali)~/linux-lab$ cat notes.txt
linus fundamental
kali linux
(kali@kali)~/linux-lab$ ls -la
total 12
drwxrwxr-x  2 kali kali 4096 Oct  5 07:08 .
drwx----- 18 kali kali 4096 Oct  5 07:03 ..
-rw-rw-r--  1 kali kali   29 Oct  5 07:08 notes.txt
```

Step 3 — Searching with `find` and `grep`

Commands

```
mkdir files
echo "password-testing" > files/config.txt
cat files/config.txt
echo "hello" > files/config.txt
find . -name "config.txt"      # first attempt, quote problem (see Troubleshooting #2)
find . -name "config.txt"      # second attempt
```

```
grep -r "password" .
```

What I did. I created a `files` subdirectory and a `config.txt` containing the dummy string `password-testing`, then read it back with `cat`. I then overwrote the file with `hello` using `>`. After that I tried to locate `config.txt` with `find`, and prepared a recursive `grep` for the word `password`.

Result

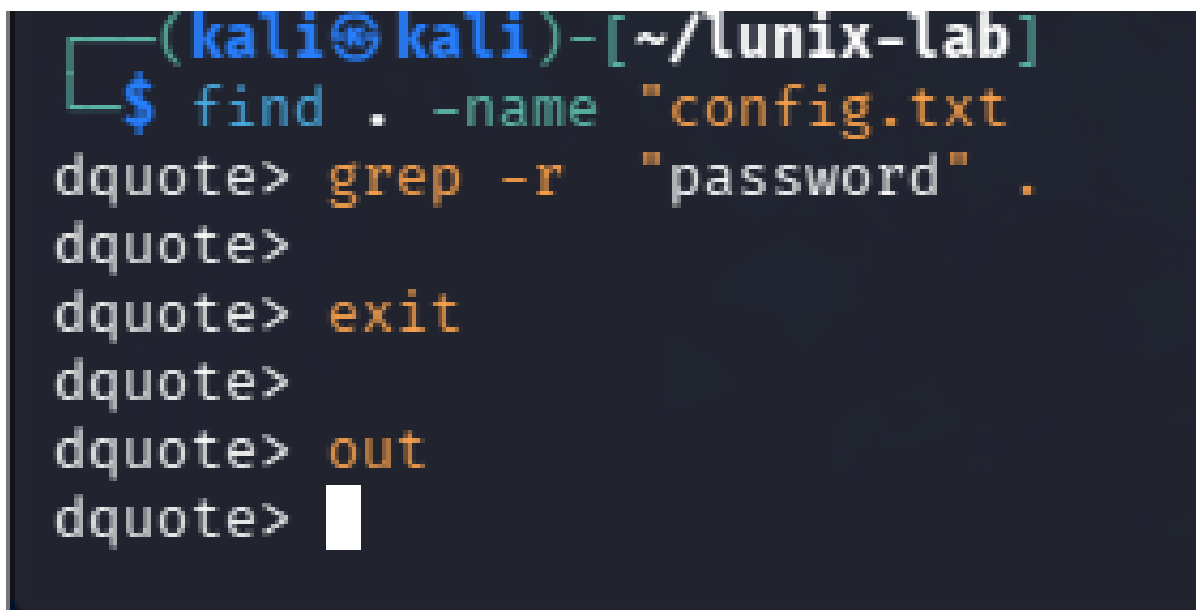
- `cat files/config.txt` printed the dummy string (the left edge of the screenshot is cropped, so it reads `assword-testing`).
- The first `find` did not execute. The shell showed a `dquote> / quote>` continuation prompt, and the lines I typed afterwards (`grep -r "password" .`, `exit`, `out`) went into the pending command instead of running.
- The second `find . -name "config.txt"` returned the file's path. The left edge is cropped, so only `/files/config.txt` is visible, which corresponds to `./files/config.txt`.
- The `grep -r "password" .` command is visible at the bottom of the last screenshot, but **its output is not captured**. Since `config.txt` had been overwritten with `hello` before the search, I cannot say from the screenshots what the search returned.

Options used

- `find . -name "config.txt"`: `.` is the starting directory, `-name` matches by filename.
- `grep -r "password" .`: `-r` searches recursively through the directory tree.

What I learned: how `find` and `grep` complement each other (find by name, grep by content), that `>` destroys the previous content of a file, and how unbalanced quotes affect the shell.

Evidence: `Screenshot_2026-10-05_141347.png`, `Screenshot_2026-10-05_141706.png`



```
(kali@kali)-[~/linux-lab]
$ find . -name "config.txt
dquote> grep -r "password" .
dquote>
dquote> exit
dquote>
dquote> out
dquote> 
```

```
-(kali㉿kali)-[~/linux-lab]
$ mkdir files

-(kali㉿kali)-[~/linux-lab]
$ echo "password-testing" > files/config.txt

-(kali㉿kali)-[~/linux-lab]
$ cat files/config.txt
password-testing

-(kali㉿kali)-[~/linux-lab]
$ echo "hello" > files/config.txt

-(kali㉿kali)-[~/linux-lab]
$ find . -name "config.txt"
.
files/config.txt

quote> grep -r "password" .
quote>
quote> exit
quote>
quote> out
quote>
```

Step 4 — Changing file permissions with `chmod`

Commands

```
echo "secret data " > secret.txt
ls -l secret.txt
chmod 600 secret.txt
ls -l secret.txt
chmod 644 secret.txt
ls -l secret.txt
chmod 755 secret.txt
ls -l secret.txt
```

What I did. I created `secret.txt` containing the dummy text `secret data` and applied three octal permission modes in turn, checking the result with `ls -l` each time.

Result (the leading file-type character is cropped in the screenshot):

Command	Resulting permissions	Meaning
(default after creation)	<code>rw-rw-r--</code>	owner and group read/write, others read
<code>chmod 600</code>	<code>rw-----</code>	owner read/write only
<code>chmod 644</code>	<code>rw-r--r--</code>	owner read/write, everyone else read
<code>chmod 755</code>	<code>rxwxr-xr-x</code>	owner full, group and others read/execute

The file is 13 bytes (12 characters plus a newline). After `chmod 755` the filename is displayed in green, which is the terminal's colour for an executable file.

What I learned: each octal digit is a sum of read (4), write (2) and execute (1), applied in the order owner, group, others. Restricting a sensitive file to `600` keeps other users from reading it. Granting execute makes a file runnable and changes how `ls` colours it.

Evidence: `Screenshot_2026-10-05_141856.png`

```

(kali㉿kali)-[~/linux-lab]
$ echo "secret data" > secret.txt

(kali㉿kali)-[~/linux-lab]
$ ls -l secret.txt
-rw-rw-r-- 1 kali kali 13 Oct  5 07:16 secret.txt

(kali㉿kali)-[~/linux-lab]
$ chmod 600 secret.txt

(kali㉿kali)-[~/linux-lab]
$ ls -l secret.txt
-rw----- 1 kali kali 13 Oct  5 07:16 secret.txt

(kali㉿kali)-[~/linux-lab]
$ chmod 644 secret.txt

(kali㉿kali)-[~/linux-lab]
$ ls -l secret.txt
-rw-r--r-- 1 kali kali 13 Oct  5 07:16 secret.txt

(kali㉿kali)-[~/linux-lab]
$ chmod 755 secret.txt

(kali㉿kali)-[~/linux-lab]
$ ls -l secret.txt
-rwxr-xr-x 1 kali kali 13 Oct  5 07:16 secret.txt

```

Step 5 — Background jobs and process inspection

Commands

```

ping -i 10 127.0.0.1 &
jobs
ps aux | grep ping
kill 127.0.0.1      # fails, see Troubleshooting #3
jobs

```

What I did. I started a ping against the loopback address as a background job, then inspected it with `jobs` and `ps aux | grep ping`. I attempted to terminate it with `kill`.

Result

- The shell printed the job number and PID ([1] 10637) and returned the prompt immediately.
- Ping output kept printing into the terminal while the job ran, interleaving with what I typed. The header read `PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.` followed by replies such as `64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=2.08 ms`. Later replies showed times of roughly 0.03 to 0.07 ms.
- `jobs` showed `[1] + running ping -i 10 127.0.0.1`.
- `ps aux | grep ping` listed two processes:

```

kali 10637 0.0 0.1 7944 2912 pts/0 SN 07:19 0:00 ping -i 10 127.0.0.1
kali 15748 0.0 0.1 6552 2456 pts/0 S+ 07:29 0:00 grep --color=auto ping

```

- The `kill` attempt failed (see Troubleshooting #3). A later `jobs` still showed the job as running.
- Near the bottom of the last screenshot there is an interleaved line containing `ping -i 10 127.0.0.1 &` followed by `jobs`. Because ping output was scrolling over my input, I cannot tell from the screenshot what I was typing there [*partially unreadable in screenshot*].
- **The screenshots do not show the ping job being successfully stopped.**

Options used

- `ping -i 10`: wait 10 seconds between packets. The sequence numbers reaching the 60s over a start time of 07:19 and a `ps` timestamp of 07:29 are consistent with that interval.
- `&`: run the command in the background.
- `ps aux`: list all processes with user, PID, CPU/memory use, state and start time.
- `grep ping`: filter that list. The `grep` process itself appears in its own results, which is expected.

Output analysis: the process state `SN` means interruptible sleep with a low-priority (nice) flag. `S+` for the `grep` process means sleeping in the foreground process group. Replies from `127.0.0.1` with `ttl=64` and sub-millisecond latency are normal for traffic that never leaves the machine.

What I learned: the difference between a job number ([1]) and a PID (10637), that `kill` needs a process or job identifier, and that background output can clutter an interactive terminal.

Evidence: [Screenshot_2026-10-05_143037.png](#), [Screenshot_2026-10-05_143102.png](#)

```
(kali@kali)-[~/linux-lab]
$ ping -i 10 127.0.0.1 &
[1] 10637
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
(kali@kali)-[~/linux-lab]
$ 64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=2.08 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.036 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.038 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.037 ms
64 bytes from 127.0.0.1: icmp_seq=5 ttl=64 time=0.040 ms
64 bytes from 127.0.0.1: icmp_seq=6 ttl=64 time=0.037 ms
64 bytes from 127.0.0.1: icmp_seq=7 ttl=64 time=0.046 ms
64 bytes from 127.0.0.1: icmp_seq=8 ttl=64 time=0.038 ms
64 bytes from 127.0.0.1: icmp_seq=9 ttl=64 time=0.042 ms
64 bytes from 127.0.0.1: icmp_seq=10 ttl=64 time=0.039 ms
64 bytes from 127.0.0.1: icmp_seq=11 ttl=64 time=0.038 ms
64 bytes from 127.0.0.1: icmp_seq=12 ttl=64 time=0.035 ms
64 bytes from 127.0.0.1: icmp_seq=13 ttl=64 time=0.038 ms
64 bytes from 127.0.0.1: icmp_seq=14 ttl=64 time=0.036 ms
64 bytes from 127.0.0.1: icmp_seq=15 ttl=64 time=0.037 ms
64 bytes from 127.0.0.1: icmp_seq=16 ttl=64 time=0.061 ms
64 bytes from 127.0.0.1: icmp_seq=17 ttl=64 time=0.028 ms
64 bytes from 127.0.0.1: icmp_seq=18 ttl=64 time=0.076 ms
64 bytes from 127.0.0.1: icmp_seq=19 ttl=64 time=0.036 ms
64 bytes from 127.0.0.1: icmp_seq=20 ttl=64 time=0.036 ms
64 bytes from 127.0.0.1: icmp_seq=21 ttl=64 time=0.041 ms
64 bytes from 127.0.0.1: icmp_seq=22 ttl=64 time=0.036 ms
64 bytes from 127.0.0.1: icmp_seq=23 ttl=64 time=0.037 ms
64 bytes from 127.0.0.1: icmp_seq=24 ttl=64 time=0.039 ms
64 bytes from 127.0.0.1: icmp_seq=25 ttl=64 time=0.038 ms
64 bytes from 127.0.0.1: icmp_seq=26 ttl=64 time=0.036 ms
64 bytes from 127.0.0.1: icmp_seq=27 ttl=64 time=0.037 ms
64 bytes from 127.0.0.1: icmp_seq=28 ttl=64 time=0.065 ms
64 bytes from 127.0.0.1: icmp_seq=29 ttl=64 time=0.037 ms
64 bytes from 127.0.0.1: icmp_seq=30 ttl=64 time=0.035 ms
64 bytes from 127.0.0.1: icmp_seq=31 ttl=64 time=0.036 ms
64 bytes from 127.0.0.1: icmp_seq=32 ttl=64 time=0.037 ms
64 bytes from 127.0.0.1: icmp_seq=33 ttl=64 time=0.038 ms
64 bytes from 127.0.0.1: icmp_seq=34 ttl=64 time=0.032 ms
64 bytes from 127.0.0.1: icmp_seq=35 ttl=64 time=0.039 ms
64 bytes from 127.0.0.1: icmp_seq=36 ttl=64 time=0.038 ms
64 bytes from 127.0.0.1: icmp_seq=37 ttl=64 time=0.036 ms
```

```
(kali@kali)-[~/linux-lab]
$ 64 bytes from 127.0.0.1: icmp_seq=58 ttl=64 time=0.041 ms
(kali@kali)-[~/linux-lab]
$ job64 bytes from 127.0.0.1: icmp_seq=59 ttl=64 time=0.035 ms
jobs
[1] + running ping -i 10 127.0.0.1
(kali@kali)-[~/linux-lab]
$ ps aux 64 bytes from 127.0.0.1: icmp_seq=60 ttl=64 time=0.044 ms
| grep 64 bytes from 127.0.0.1: icmp_seq=61 ttl=64 time=0.038 ms
ping
kali 10637 0.0 0.1 7944 2912 pts/0 SN 07:19 0:00 ping -i 10 127.0.0.1
kali 15748 0.0 0.1 6552 2456 pts/0 S+ 07:29 0:00 grep --color=auto ping
(kali@kali)-[~/linux-lab]
$ kill 64 bytes from 127.0.0.1: icmp_seq=62 ttl=64 time=0.044 ms
127.0.0.1
kill: illegal pid: 127.0.0.1
(kali@kali)-[~/linux-lab]
$ 64 bytes from 127.0.0.1: icmp_seq=63 ttl=64 time=0.036 ms
jobs
[1] + running ping -i 10 127.0.0.1
(kali@kali)-[~/linux-lab]
$ 64 bytes from 127.0.0.1: icmp_seq=64 ttl=64 time=0.036 ms
ping -i 10 127.0.0.1 & 64 bytes from 127.0.0.1: icmp_seq=65 ttl=64 time=0.031 ms
64 bytes from 127.0.0.1: icmp_seq=66 ttl=64 time=0.311 ms
jobs64 bytes from 127.0.0.1: icmp_seq=67 ttl=64 time=0.055 ms
```

Step 6 — Gathering system information

Commands

```
id
uname -a
free -h
df -h
lspci
lsusb
dmesg
```

Result

- `id: uid=1000(kali) gid=1000(kali)` with membership in several groups, including `sudo`, `wireshark`, `kaboxer`, `vboxsf`, `netdev`, `plugdev` and `lpadmin`. One group in the middle of the list is cut off by the screen edge.
- `uname -a: Linux kali 6.19.14+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.14-1+kali1 (2026-05-05) x86_64 GNU/Linux.`
- `free -h:`

```

total    used    free   shared  buff/cache   available
Mem:     1.9Gi  868Mi  252Mi   12Mi     1.0Gi      1.1Gi
Swap:    1.1Gi    0B    1.1Gi

```

- `df -h:` the root filesystem `/dev/sda1` is 19G with 15G used and 3.0G available, at **84%** usage. The rest are virtual filesystems (`udev`, `tmpfs`, etc.).
- `lspci:` virtualised hardware typical of VirtualBox, including an Intel 440FX host bridge, a VMware SVGA II display adapter, an Intel 82540EM Gigabit Ethernet controller, a VirtualBox Guest Service device, an AC'97 audio controller and a SATA AHCI controller.
- `lsusb:` two Linux Foundation root hubs and a `VirtualBox USB Tablet`.
- `dmesg:` only the first kernel line is visible, confirming the kernel version `6.19.14+kali-amd64` and the compiler (`x86_64-linux-gnu-gcc-15`). The rest is cut off.

Options used

- `uname -a:` print all system information.
- `free -h` and `df -h:` `-h` prints sizes in human-readable units.

What I learned: how to confirm user identity and privileges (`id`), identify the kernel and architecture, check memory and disk headroom, and recognise that the PCI/USB device list reflects a virtual machine rather than physical hardware. The 84% root usage is a practical reminder to watch disk space in a VM.

Evidence: `Screenshot_2026-10-05_143425.png`

```

kali@kali: ~
$ id
uid=1000(kali) gid=1000(kali) groups=1000(kali),4(sdm),20(dialout),26(cdrom),25(floppy),27(sudo),29(audio),30(dia),44(o),46(plugdev),100(users),101(netdev),102(scanner),114(wireshark),119(kaboxer),968(vboxsf),979(bluetooth),999(lpadmin)

kali@kali: ~
$ uname -a
Linux kali 6.19.14+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.14-1+kali1 (2026-05-05) x86_64 GNU/Linux

kali@kali: ~
$ free -h
              total        used        free      shared  buff/cache   available
Mem:           1.9Gi       868Mi       252Mi        12Mi     1.0Gi       1.1Gi
Swap:           1.1Gi          0B           1.1Gi

kali@kali: ~
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            951M   0 951M   0% /dev
tmpfs           197M  988K  196M   1% /run
/dev/sda1       19G   15G   3.0G  84% /
tmpfs           985M  4.0K  985M   1% /dev/shm
none            1.0M   0 1.0M   0% /run/credentials/systemd-journald.service
tmpfs           985M  8.0K  985M   1% /tmp
none            1.0M   0 1.0M   0% /run/credentials/getty@tty1.service
tmpfs           197M  108K  197M   1% /run/user/1000

kali@kali: ~
$ lspci
00:00.0 Host bridge: Intel Corporation 440FX - 82441FX PMC [Natoma] (rev 02)
00:01.0 ISA bridge: Intel Corporation 82371SB PIIX3 ISA [Natoma/Triton II]
00:01.1 IDE interface: Intel Corporation 82371AB/EB/MB PIIX4 IDE (rev 01)
00:02.0 VGA compatible controller: VMware SVGA II Adapter
00:03.0 Ethernet controller: Intel Corporation 82540EM Gigabit Ethernet Controller (rev 02)
00:04.0 System peripheral: InnoTek Systemberatung GmbH VirtualBox Guest Service
00:05.0 Multimedia audio controller: Intel Corporation 82801AA AC'97 Audio Controller (rev 01)
00:06.0 USB controller: Apple Inc. KeyLargo/Intrepid USB
00:07.0 Bridge: Intel Corporation 82371AB/EB/MB PIIX4 ACPI (rev 08)
00:0b.0 USB controller: Intel Corporation 82801FB/FR/FW/FRW (ICH6 Family) USB2 EHCI Controller (rev 02)
00:0d.0 SATA controller: Intel Corporation 82801HM/HEM (ICH8M/ICH8M-E) SATA Controller [AHCI mode] (rev 02)

kali@kali: ~
$ lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 002 Device 001: ID 1d6b:0001 Linux Foundation 1.1 root hub
Bus 002 Device 002: ID 80ee:0021 VirtualBox USB Tablet

kali@kali: ~
$ dmesg
[ 0.000000] Linux version 6.19.14+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-17) 15.2.0, G

```

Step 7 — Writing and running a Bash system-info script

Commands

```

chmod +x linux.sh          # fails
ls
chmod +x linux.sh.save     # fails
chmod +x linux.sh          # fails
chmod +x linux.sh.save     # succeeds
./linux.sh.save

```

```
nano linux.sh.save
```

What I did. From my home directory I tried to make a script executable, but kept hitting "No such file or directory". After listing the directory I found the file was actually named `linux.sh.save`. I applied `chmod +x` to that name, ran it, and then opened it in `nano` to inspect the contents.

Script contents visible in nano 9.0 (reproduced as shown, including typos):

```
#!/bin/bash
echo"iduser"
id
echo
echo "host"
hostname
echo
echo "kernel"
uname -a
echo
echo "c.directury"
pwd
echo "memory"
free -h
echo
echo "disk"
df -h
echo "process"
ps aux
echo
echo "pci dev"
lspci
echo
echo "usv dev"
lsusb
```

Result of running it

```
./linux.sh.save: line 2: echoiduser: command not found
uid=1000(kali) gid=1000(kali) groups=1000(kali),4(adm),20(dialout),...
(blank line)
host
kali
```

The remaining output is cut off in the screenshot. Everything after the `hostname` result is therefore not documented here.

Output analysis: the script runs top to bottom. Line 2 is `echo"iduser"` with no space, so Bash read `echoiduser` as a single command name, which doesn't exist. The following `id` and `hostname` lines ran normally, showing that one failed line does not stop the script. The editor screenshot shows the missing space and the labels `c.directury` and `usv dev` still uncorrected at that moment. **Whether I fixed them afterwards is not shown.**

What I learned: a script needs a shebang (`#!/bin/bash`) and the execute bit; `command not found` can come from a typo in spacing; and error messages include a line number that points straight at the faulty line.

Evidence: [Screenshot_2026-10-05_150032.png](#), [Screenshot_2026-10-05_150256.png](#)

```

(kali@kali)-[~]
$ chmod +x linux.sh
chmod: cannot access 'linux.sh': No such file or directory

(kali@kali)-[~]
$ ls
Desktop Documents Downloads linux.sh.save linux-lab Music Pictures Projects Public Templates Videos

(kali@kali)-[~]
$

(kali@kali)-[~]
$ chmod +x linux.sh.save
chmod: cannot access 'linux.sh.save': No such file or directory

(kali@kali)-[~]
$ chmod +x linux.sh
chmod: cannot access 'linux.sh': No such file or directory

(kali@kali)-[~]
$ chmod +x linux.sh
chmod: cannot access 'linux.sh': No such file or directory

(kali@kali)-[~]
$ chmod +x linux.sh.save

(kali@kali)-[~]
$ ./linux.sh.save
./linux.sh.save: line 2: echoiduser: command not found
uid=1000(kali) gid=1000(kali) groups=1000(kali),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(v
o),46(plugdev),100(users),101(netdev),102(scanner),114(wireshark),119(kaboxer),968(vboxsf),979(bluetooth),999(lpadmin)
host
kali

```

```

Session Actions Edit View Help
GNU nano 9.0 linux.sh.save
#!/bin/bash
echo "iduser"
id
echo
echo "host"
hostname
echo "kernel"
uname -a
echo
echo "c.directory"
pwd
echo "memory"
free -h
echo
echo "disk"
df -h
echo "process"
ps aux
echo
echo "pci dev"
lspci
echo
echo "usb dev"
lsusb

```

Troubleshooting

1. cd linux-lab when already inside it

- **Problem:** cd: no such file or directory: linux-lab
- **Cause:** I was already in ~/linux-lab (visible in the prompt), so there is no linux-lab subdirectory there.
- **Resolution:** I carried on from the current directory without further changes.
- **Lesson:** check the prompt before using a relative path. Relative paths are resolved from the current directory.

2. Unterminated quote in find

- **Problem:** after the first find . -name "config.txt", the shell showed dquote> / quote> prompts and swallowed my following commands (grep ..., exit, out).
- **Cause:** the shell considered a double quote unclosed. The exact reason (for example a mismatched or stray quote character) cannot be confirmed from the screenshot alone.

- **Resolution:** I then retyped `find . -name "config.txt"` and it returned `./files/config.txt`. How I left the continuation prompt (likely Ctrl+C) is not visible.
- **Lesson:** a continuation prompt means the shell is waiting for more input. Interrupt it and retype the command rather than continuing to type.

3. kill with an IP address instead of a PID

- **Problem:** `kill: illegal pid: 127.0.0.1`
- **Cause:** `kill` takes a PID or job spec, such as `10637` or `%1`. I passed the ping target address, so it was rejected.
- **Resolution:** a later `jobs` call still showed the job running. The screenshots do not show a successful termination.
- **Lesson:** use `ps` or `jobs` to find the PID or job number first, then `kill <PID>` or `kill %1`.

4. chmod on files that did not exist, and a missing space in the script

- **Problem:** repeated `chmod: cannot access '...': No such file or directory`, then `echo: iduser: command not found`.
- **Cause:** I typed names that didn't exist (`linux.sh`, `linux.sh`, `linux.sh.save`). The real file was `linux.sh.save`, as shown by `ls`. In the script, `echo"iduser"` lacks a space after `echo`.
- **Resolution:** `ls` revealed the real filename, after which `chmod +x linux.sh.save` succeeded. The script typo was visible in `nano`; a correction is not shown.
- **Lesson:** list the directory before guessing filenames. Read error messages literally: they name the exact file or line that failed. The `.save` suffix is commonly created by `nano` as an emergency save, but the screenshots do not confirm how this file came about.

Security & Privacy Review

Every screenshot was checked for credentials, tokens, keys, public or private IP addresses (other than loopback), MAC addresses, real names, email addresses, hostnames, SSIDs and personal paths.

- `password-testing` and `secret data` are dummy strings I wrote for the exercise. They are not real credentials.
- `127.0.0.1` is the loopback address and is not sensitive.
- The username and hostname `kali` are the Kali defaults.
- `devel@kali.org` in the `dmesg` line is the Kali kernel build contact, not a personal address.
- PCI/USB device names are generic VirtualBox virtual hardware, not unique identifiers.
- The kernel version and group list reveal nothing exploitable.

Conclusion: no redaction is required in any of the 10 screenshots. They can be published unmodified.

Screenshot Evidence

#	File	What It Shows	Important Evidence	Sensitive Information	Action
01	<code>Screenshot_2026-10-05_140435.png</code>	VirtualBox window, terminal in ~	<code>pwd</code> , <code>ls</code> , <code>mkdir</code> , <code>cd</code> , <code>cd ..</code>	VM name <code>kali_LINUX</code> (generic)	Keep
02	<code>Screenshot_2026-10-05_140944.png</code>	File creation, redirection, copy, move	<code>cd</code> error, <code>></code> , <code>>></code> , <code>cp</code> , <code>mv</code> , <code>ls -la</code>	None	Keep
03	<code>Screenshot_2026-10-05_141347.png</code>	Cropped view of the <code>dquote></code> problem	Unterminated quote swallowing input	None	Keep
04	<code>Screenshot_2026-10-05_141706.png</code>	<code>files/config.txt</code> , <code>find</code> , <code>grep</code>	Overwrite with <code>></code> , <code>find</code> result, <code>grep</code> command	Dummy "password" string	Keep
05	<code>Screenshot_2026-10-05_141856.png</code>	<code>chmod 600 / 644 / 755</code>	Permission strings per mode	Dummy "secret data"	Keep
06	<code>Screenshot_2026-10-05_143037.png</code>	Background <code>ping</code> start	<code>[1] 10637</code> , loopback replies	Loopback IP only	Keep

#	File	What It Shows	Important Evidence	Sensitive Information	Action
07	Screenshot_2026-10-05_143102.png	jobs, ps aux grep, failed kill	PID 10637, illegal pid error	None	Keep
08	Screenshot_2026-10-05_143425.png	System information commands	id, uname, free, df, lspci, lsusb, dmesg	None meaningful	Keep
09	Screenshot_2026-10-05_150032.png	chmod errors and script run	No-such-file errors, echoiduser error	None	Keep
10	Screenshot_2026-10-05_150256.png	Script in nano 9.0	Script source, typos visible	None	Keep

Skills Demonstrated

- Linux command-line usage and shell navigation
- File and directory management, including redirection
- Reading and changing Unix permissions
- Searching a filesystem by name and by content
- Starting, inspecting and attempting to stop background processes
- Collecting and interpreting basic host information on Kali
- Writing a simple Bash script and debugging it from error output
- Troubleshooting through error messages and directory listings

Key Takeaways

- The shell does exactly what is typed. > overwrote my file, mv replaced a file without asking, and a missing space turned echo "iduser" into a non-existent command.
- ls -l output is worth reading column by column: permissions, link count, owner, group, size, timestamp, name.
- Error messages were my best diagnostic. No such file, illegal pid and command not found each told me exactly what to fix.
- Output from background jobs lands in the active terminal, so redirecting it is worth considering in longer sessions.
- Checking df -h showed the VM's disk at 84%, which is something I should keep an eye on.

Cybersecurity Relevance

This is foundational work, not an offensive exercise. The skills it builds are still the ones other security tasks depend on:

- **Permissions:** misconfigured permissions on sensitive files are a common finding in Linux hardening and privilege-escalation reviews. Understanding 600, 644 and 755 is the starting point.
- **Searching:** find and grep -r are everyday tools for locating configuration files and for spotting strings such as hard-coded credentials, which is useful in administration, incident response and forensics.
- **Processes:** ps, jobs and kill are basic tools for identifying and stopping unexpected activity on a host.
- **System enumeration:** id, uname, lspci and similar commands tell you what you are working on and what privileges you hold, which is a routine first step in both administration and assessment.
- **Scripting:** automating repeated checks with Bash saves time and makes results repeatable.

Conclusion

In this lab I worked through Linux navigation, file handling, searching, permissions, process management, system information and a first Bash script on a Kali VM. Several commands failed, and working out why (wrong directory, unbalanced quote, wrong kill argument, misspelled filenames, a missing space in a script) taught me as much as the commands that worked. A few results are not captured in the screenshots: the output of the grep search, the end of the script's output, and whether the ping job was

eventually stopped. I have noted these rather than filling them in. The lab adds to my KLCP/PEN-103 study notes as a record of basic Linux proficiency that later chapters build on.